Reporte de Laboratorio

Despliegue Continuo de API REST Express en Render (Cloud)

Asignatura: Desarrollo Web	**Fecha:** 21/10/2026
Repositorio GitHub: https://github.com/lpinedar8-afk/Tarea-desarrollo-web	**Entorno:** Producción (Render Free Tier)
URL de Servicio Render: https://tarea-desarrollo-web.onrender.com	**NODE_ENV:** production

1. Configuración de Archivos del Proyecto

Se estructuró el proyecto Node.js/Express incorporando buenas prácticas de seguridad e infraestructura:

Archivo	Propósito	Estado
`.env`	Almacenamiento de variables locales (PORT, NODE_ENV, FRONTEND_URL).	**OK (Ignorado)**
`.gitignore`	Excluye node_modules, .env y logs del control de versiones.	**OK (Configurado)**
`package.json`	Declara dependencias (express, cors, dotenv) y scripts start/dev.	**OK (Válido)**
`index.js`	Servidor Express con endpoints /, /health y /api/productos.	**OK (Ejecutando)**
`postman_collection.json`	Colección Postman con todas las solicitudes y pruebas automatizadas.	**OK (Documentado)**

2. Repositorio Git y Control de Versiones

Se inicializó el repositorio local con Git en la rama **main**. El archivo .env quedó efectivamente fuera del seguimiento de Git para evitar filtrar credenciales. El repositorio fue publicado en GitHub en la URL: https://github.com/lpinedar8-afk/Tarea-desarrollo-web.

Parámetro	Valor
Rama principal	`main`
Commit inicial (v1)	`d11921a - feat: inicializar API Express`
Commit actualización (v2)	`e7dd027 - feat: actualizar API y soporte de consulta por ID`

3. Despliegue en Render (Web Service Free Tier)

Se configuró el Web Service en Render vinculado al repositorio de GitHub con los siguientes parámetros:

Configuración	Valor Establecido	Descripción
Environment	`Node`	Entorno de ejecución Node.js
Plan / Instance Type	`Free Tier`	Instancia gratuita de Render
Build Command	`npm install`	Instala las dependencias declaradas
Start Command	`npm start`	Ejecuta node index.js
Variable de Entorno	`NODE_ENV = production`	Configurada en el panel de Render

4. Evidencia de Pruebas Postman en Producción (Versión 1)

Se ejecutaron pruebas HTTP mediante Postman sobre la URL pública de producción **https://tarea-desarrollo-web.onrender.com**, comprobando la respuesta real de la nube y confirmando que NODE_ENV refleja production.

Método y Ruta	Status	Respuesta JSON (Producción Render)
GET /	200 OK	{"mensaje": "Bienvenido a la API, version 2", "entorno": "production"}
GET /health	200 OK	{"status": "ok", "servicio": "API Desarrollo Web", "timestamp": "2026-10-21T07:22:49.725Z"}
GET /api/productos	200 OK	[{"id": 1, "nombre": "Teclado", "precio": 250}, {"id": 2, "nombre": "Mouse", "precio": 125}]
POST /api/productos	201 Created	{"mensaje": "Solicitud de creación validada", "producto": {"nombre": "Monitor Gamer 24 pulgadas", "precio": 1350}, "persistido": false}
POST /api/productos	400 Bad Request	{"error": "Datos de producto inválidos"}
GET /api/inexistente	404 Not Found	{"error": "Ruta no encontrada"}

5. Actualización de Código y Despliegue Automático (CI/CD)

Siguiendo los requerimientos del laboratorio, se aplicó un cambio al código fuente: se agregó el endpoint para consultar productos por ID (GET /api/productos/:id) y se actualizó el mensaje de bienvenida a la versión 2.1. Al realizar git commit y git push origin main, Render detectó automáticamente el nuevo commit y desplegó la nueva versión sin intervención manual.

Método y Ruta	Status	Respuesta Actualizada (Producción Render)
GET /	200 OK	{"mensaje": "Bienvenido a la API, version 2.1 (Produccion)", "entorno": "production"}
GET /api/productos/1	200 OK	{"id": 1, "nombre": "Teclado", "precio": 250}
GET /api/productos/99	404 Not Found	{"error": "Producto no encontrado"}

6. Conclusiones y Resumen

1. **Protección de Secretos:** El archivo .env quedó protegido mediante .gitignore, asegurando que información sensible nunca se almacene en el repositorio público.

2. **Variables de Entorno en la Nube:** La variable NODE_ENV=production se inyectó directamente desde la configuración de Render, demostrando la separación entre código y configuración recomendada por Twelve-Factor App.

3. **Integración y Despliegue Continuo (CI/CD):** La vinculación de GitHub con Render habilita el despliegue automático ante cada git push, reduciendo tiempos de publicación y garantizando consistencia en el entorno productivo.

4. **Validación de Rutas y Métodos HTTP:** Se validó el manejo adecuado de códigos de estado HTTP (200 OK, 201 Created, 400 Bad Request, 404 Not Found) mediante pruebas funcionales de Postman.